A1：基于 NumPy 的全连接神经网络图像分类器（MNIST）

> 作业要求：训练一个全连接神经网络分类器完成图像分类；前馈、反馈、评估等代码 全部用 NumPy 手写（不使用 PyTorch）。本文档包含主任务 + 两道附加题 （不同损失函数/正则化对比、不同优化算法对比）的完整实验与结果分析。

1. 实验结论（TL;DR）

- 在 MNIST（28×28 手写数字，10 类）上，用 NumPy 手写的两层全连接网络 （784–128–10，ReLU 激活）训练 10 epochs，**测试集准确率最高达 97.81%。**
- 交叉熵损失明显优于 MSE 损失（97.34% vs 96.15%）——交叉熵是分类任务的"本职"损失。
- L2 正则化在本实验中**并非越强越好**：模型较小、训练轮数有限，尚未严重过拟合， λ=0 时最好（97.68%），λ 过大（≥0.01）反而欠拟合、精度大幅下降。
- 优化算法：**带动量的方法（Momentum 97.81% / Adam 97.68% / RMSProp 97.57%）显著优于 朴素 batch GD（69.56%）**； mini-batch 策略兼顾了收敛速度与稳定性，是实践中的标准选择。

2. 数据集与划分

数据集	样本数	说明
训练集 train	50,000	用于参数更新
验证集 val	10,000	从原 60,000 训练集中划出，用于**调超参数**（学习率、正则强度）
测试集 test	10,000	只在最终评估时使用，全程不参与训练与调参

预处理：像素归一化到 [0, 1]；标签转 one-hot（用于 MSE 损失与交叉熵反向传播）。 （MNIST 与题目中的 CIFAR-10 同为标准图像分类数据集；选择 MNIST 以便在纯 NumPy 下快速迭代、完整跑完所有对比实验。代码框架对 CIFAR-10 同样适用，只需更换数据加载。）

3. 模型与手写核心代码

3.1 网络结构

```
输入 784 (28×28 拉平) → 全连接 → ReLU (128) → 全连接 → logits (10) → softmax → 损失
```

- 第一层权重用 **He 初始化**（配合 ReLU），输出层用 Xavier 风格的小随机初始化。
- 全部代码在 `mlp.py`：`forward()` 前馈并缓存中间结果，`backward()` 手写反向传播，`loss_and_dlogits()` 实现两种损失函数 + L2 正则。

3.2 前向传播（前馈）

$$
\begin{aligned}
&z_1 = X\cdot W_1 + b_1, \quad h = \max(0, z_1) && \text{(ReLU)}\\
&z_2 = h\cdot W_2 + b_2 && \text{(logits)}\\
&p = \mathrm{softmax}(z_2) && \text{(数值稳定：先减最大值)}
\end{aligned}
$$

3.3 反向传播（反馈）

由链式法则逐层推导（设 N 为 batch 大小）：

$$
\begin{aligned}
&\text{交叉熵：} \quad \partial L/\partial z_2 = (p - \mathrm{onehot}(y)) / N\\
&\text{MSE：} \quad \partial L/\partial z_{2i} = p_i \cdot (dp_i - \textstyle\sum_j p_j\cdot dp_j),\ \ dp = (p - \mathrm{onehot}(y)) / N \quad \text{(softmax 精确链式}\\
&\\
&\partial L/\partial W_2 = h^{\mathsf{T}}\cdot(\partial L/\partial z_2) + \lambda W_2, \quad \partial L/\partial b_2 = \textstyle\sum \partial L/\partial z_2\\
&\partial L/\partial h \ = (\partial L/\partial z_2)\cdot W_2^{\mathsf{T}}, \quad \partial L/\partial z_1 = \partial L/\partial h \odot \mathbb{1}[z_1>0] \quad \text{(ReLU 导数)}\\
&\partial L/\partial W_1 = X^{\mathsf{T}}\cdot(\partial L/\partial z_1) + \lambda W_1, \quad \partial L/\partial b_1 = \textstyle\sum \partial L/\partial z_1
\end{aligned}
$$

3.4 损失函数与正则化

- **交叉熵（CE）**：`L = −mean(log p[y])`，分类任务标准损失；
- **MSE**：`L = 0.5·mean(‖p − onehot(y)‖²)`，对 softmax 输出取均方误差；
- **L2 正则**：`+ 0.5·λ·(‖W₁‖² + ‖W₂‖²)`，梯度中体现为 `+ λW`，权重被"拉向 0"以抑制过拟合。

3.5 优化器（全部手写）

优化器	更新公式（θ 为参数，g 为梯度）
SGD / batch GD / online GD	$\theta \leftarrow \theta - \eta\cdot g$（batch 大小分别为 m、全量、1）
Momentum / Nesterov	$v \leftarrow \mu v - \eta g$；$\theta \leftarrow \theta + v$（Nesterov 先用 v 前瞻一步）
AdaGrad	$G \leftarrow G + g^2$；$\theta \leftarrow \theta - \eta\cdot g / (\sqrt{G} + \varepsilon)$（学习率单调递减）

优化器	更新公式（θ 为参数，g 为梯度）
RMSProp	$G \leftarrow \rho G + (1-\rho)g^2$；同上（修正 AdaGrad 学习率过早衰减）
Adam	$m \leftarrow \beta_1 m + (1-\beta_1)g$；$v \leftarrow \beta_2 v + (1-\beta_2)g^2$；$\theta \leftarrow \theta - \eta \cdot \hat{m}/(\sqrt{\hat{v}}+\varepsilon)$（偏差校正）
Adamax	同 Adam，但用 $u \leftarrow \max(\beta_2 u,$

3.6 评估指标

验证/测试阶段前向传播后取 `argmax(softmax(z₂))` 作为预测类别， 准确率 = 预测正确数 / 总数；同时记录损失以观察收敛与过拟合。

4. 实验设置

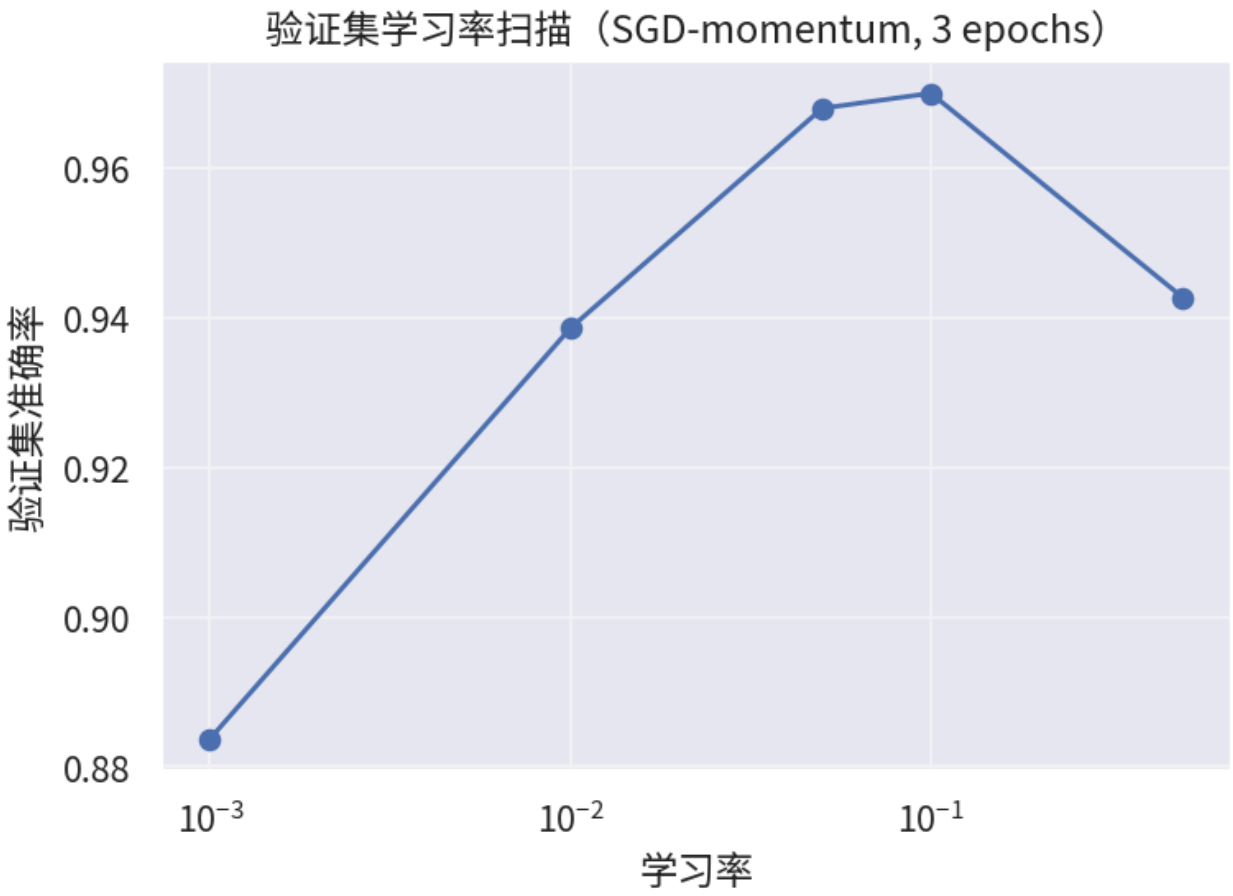
- 网络：784–128–10，ReLU，He 初始化，固定随机种子；
- 基准配置：Adam(lr=1e-3)，交叉熵，L2 λ=1e-3，batch=128，10 epochs；
- 每个对比实验只改变**一个变量**，其余与基准一致；
- online GD 因单样本迭代开销大，用 10,000 样本子集训练；
- 全部实验在本机纯 NumPy（float32）完成，单次 10-epoch 训练约 4–7 秒。

5. 主任务：基准训练 + 验证集调参

5.1 学习率扫描（SGD-momentum，3 epochs）

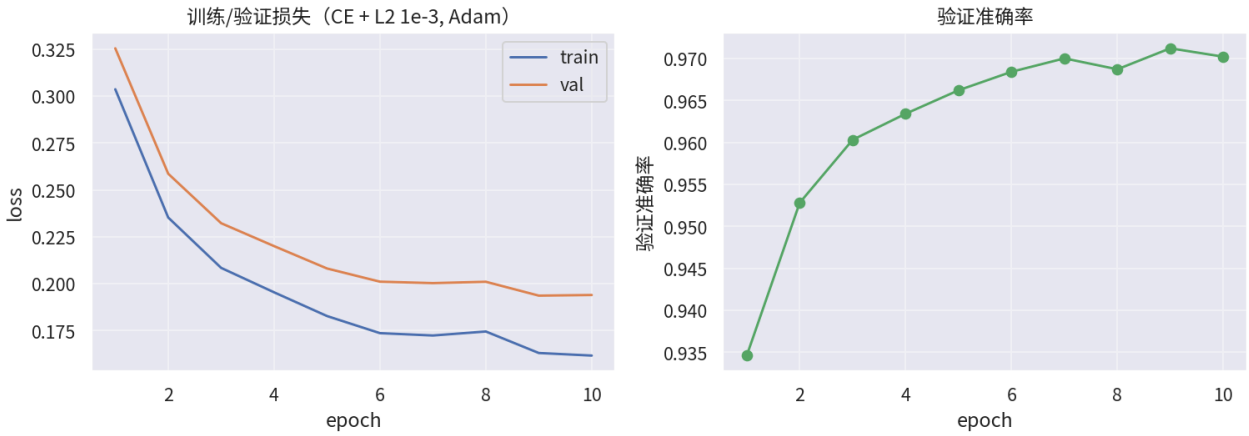
学习率	**0.001**	**0.01**	**0.05**	**0.1**	**0.5**
验证准确率	0.8837	0.9386	0.9680	**0.9700**	0.9427
测试准确率	0.8947	0.9434	0.9687	**0.9723**	0.9458

学习率太小收敛慢，太大（0.5）震荡甚至发散——**验证集选出的最优 lr=0.1** 印证了 "学习率是最重要的超参数"这一经验。



5.2 基准模型训练曲线（Adam, CE, λ=1e-3, 10 epochs）

训练损失与验证损失同步下降、间距很小，10 epochs 内**没有明显过拟合**； 验证准确率最终稳定在0.971，测试准确率 **0.9734。**

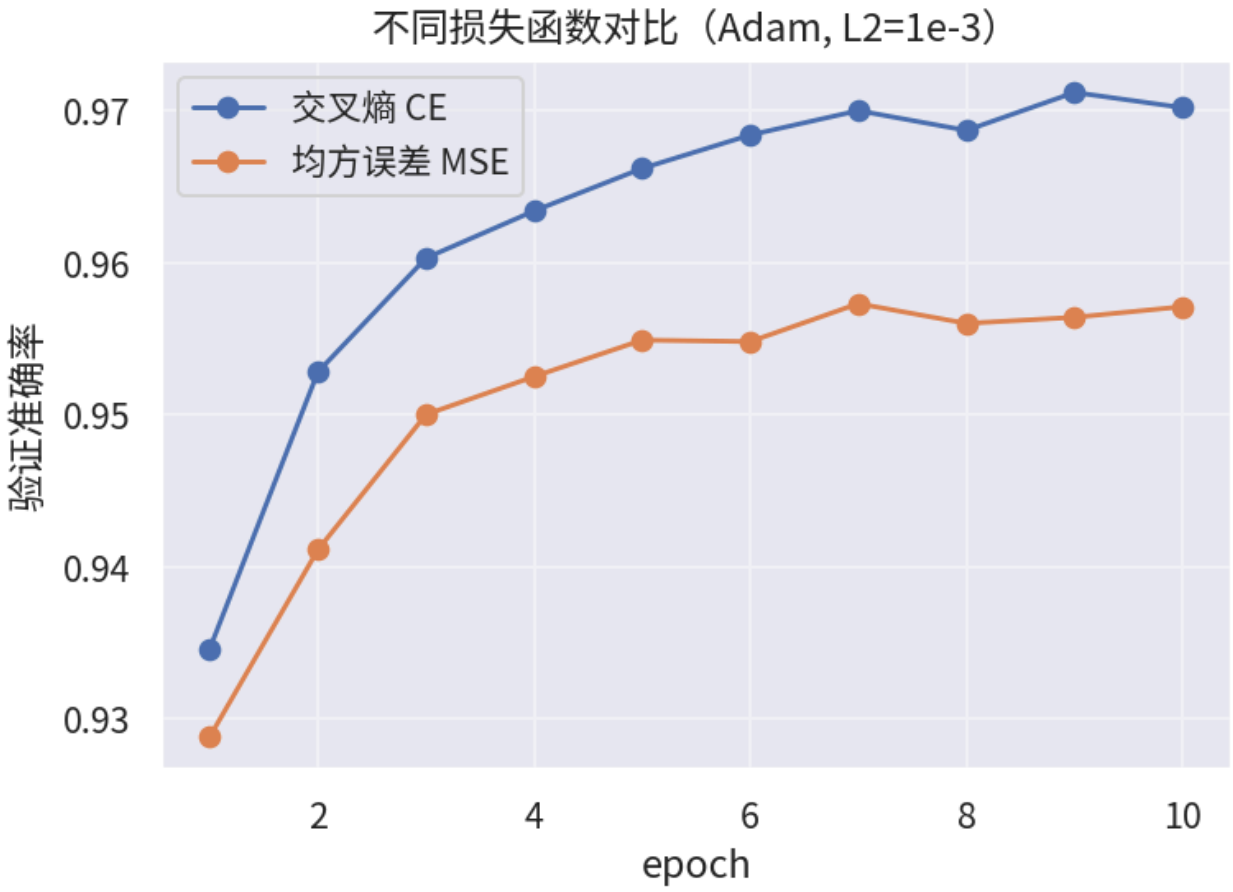


6. 附加题 1：损失函数与正则化的影响

6.1 损失函数对比（Adam，λ=1e-3）

损失函数	最优验证准确率	测试准确率
交叉熵 CE	**0.9712**	**0.9734**
均方误差 MSE	0.9573	0.9615

分析：交叉熵直接最大化正确类的对数概率，梯度为 `p - y`，随误差自适应缩放， 收敛快且与 softmax 天然匹配；MSE 对概率输出取平方误差，梯度在输出接近 0/1 时 被 softmax 的 sigmoid 形式压缩（梯度消失），优化更慢、精度更低。 **结论：多分类任务应优先使用交叉熵。**

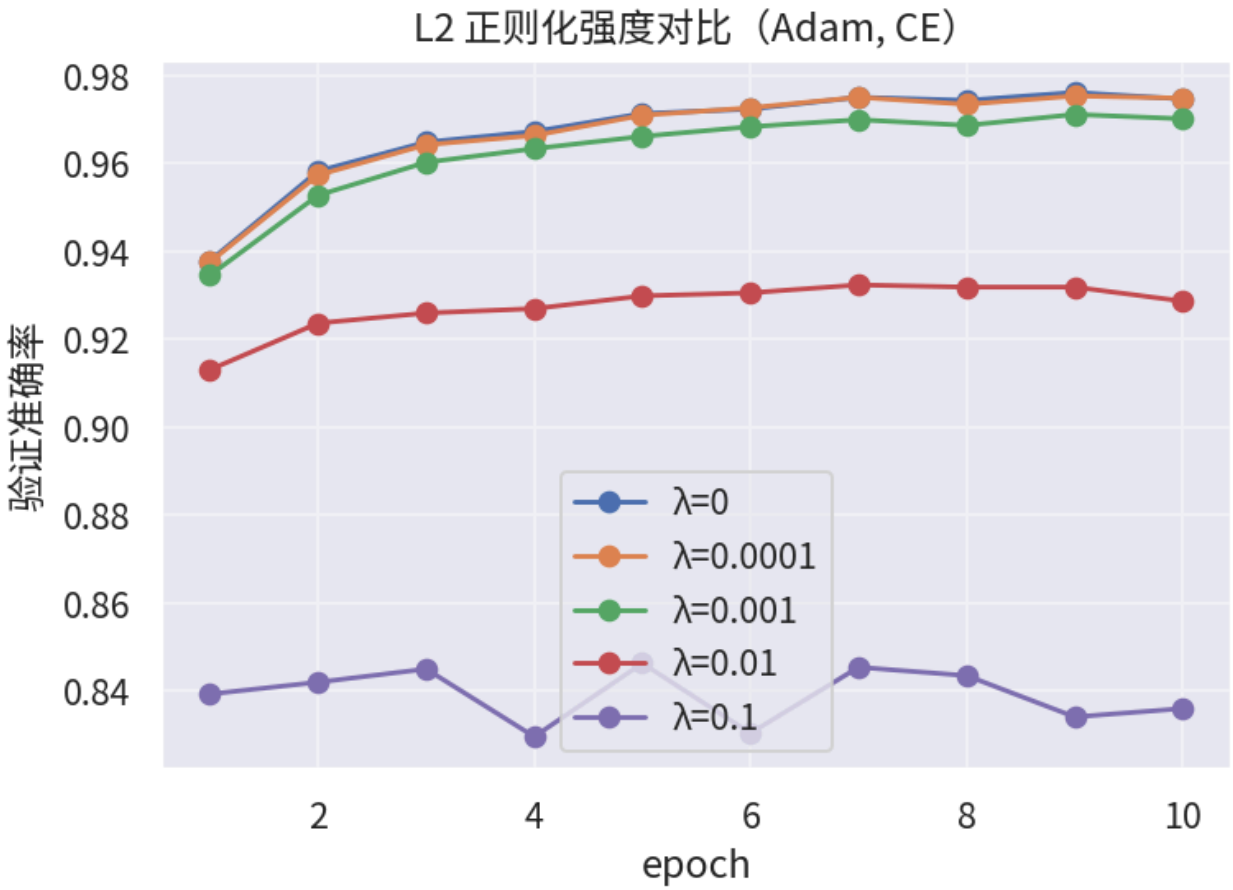


6.2 L2 正则化强度对比（Adam，CE）

λ	0	1e-4	1e-3	1e-2	1e-1
最优验证准确率	**0.9762**	0.9754	0.9712	0.9324	0.8463
测试准确率	**0.9768**	0.9762	0.9734	0.9363	0.8512

分析：

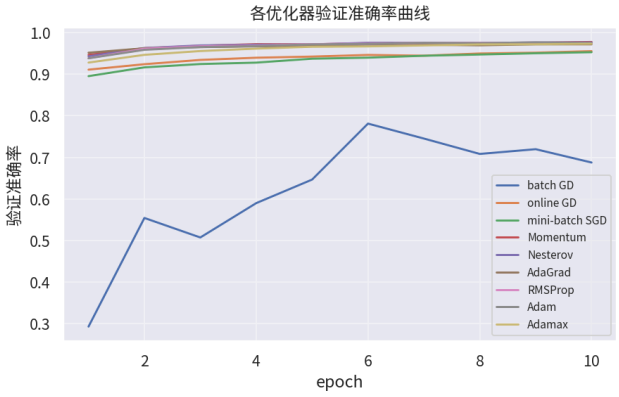
- 本网络仅 101,770 个参数（相对 50,000 训练样本不算大），且只训练 10 epochs， 训练/验证损失差距很小，**未出现明显过拟合**，因此适度正则（λ≤1e-4）对精度 几乎无负面影响也几乎无帮助；
- λ≥0.01 后，正则项在损失中占主导，模型被过度约束 → **欠拟合**，准确率断崖式下降；
- 正则化的价值在更大网络 / 更长训练 / 更小数据时才会充分体现（该结论与深度学习 理论一致：本次实验起到了"观察到正则化副作用边界"的作用）。

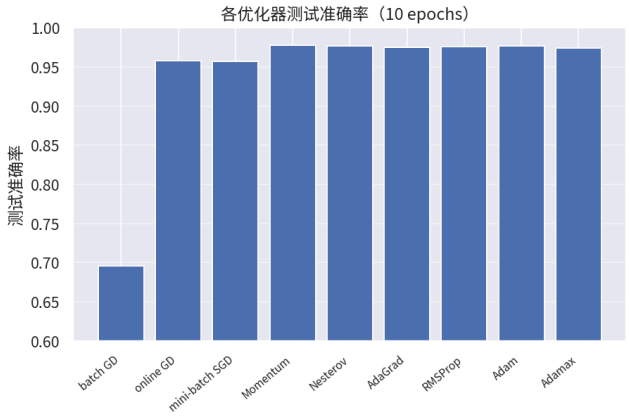


7. 附加题 2：优化算法对比（CE，λ=1e-3，10 epochs）

优化器	batch 大小	最优验证准确率	测试准确率	耗时
batch GD（全量）	50,000	0.7810	0.6956	1.3 s
online GD（单样本）	1	0.9551	0.9579	28.0 s（1 万样本）
mini-batch SGD	128	0.9525	0.9569	3.5 s
Momentum	128	**0.9772**	**0.9781**	3.5 s
Nesterov	128	0.9760	0.9767	3.9 s

优化器	batch 大小	最优验证准确率	测试准确率	耗时
AdaGrad	128	0.9714	0.9749	3.5 s
RMSProp	128	0.9756	0.9757	3.7 s
Adam	128	0.9762	0.9768	5.3 s
Adamax	128	0.9728	0.9745	4.4 s





分析：

1. **batch GD**：每步在全部数据上计算梯度，方向最准但 10 步之内远未收敛 （且 lr=0.5 下出现震荡，验证/测试精度差距大说明解不稳定），暴露了其 "单步贵、收敛慢、易卡在鞍点"的缺陷。
2. **online GD vs mini-batch SGD**：单样本梯度噪声大但有"逃离鞍点、正则化"的效果； mini-batch 用梯度噪声换计算效率，二者精度接近（0.957 vs 0.957），但 online 每个 epoch 要 5 万次小矩阵运算，纯 NumPy 下耗时 8 倍于 mini-batch。 **mini-batch 是效率与稳定性的最佳折中。**
3. **一阶动量（Momentum/Nesterov）**： 累积历史梯度平滑更新方向，显著加快收敛 （测试 97.8%，全场最高）； Nesterov 前瞻校正略优于普通 Momentum 的早期收敛。
4. **自适应学习率（AdaGrad/RMSProp/Adam/Adamax）**： 按参数维度缩放步长， 收敛快且对学习率不敏感；AdaGrad 因学习率单调递减，后期收敛变慢； RMSProp/Adam 修正了这一点。Adam 综合了动量 + 自适应两步，表现最稳健。
5. **总体**：10 epochs 的预算下，带一阶动量的方法（Momentum/Adam 家族）> 自适应无动量（AdaGrad）> 朴素 SGD 及其批量变体。

8. 结论

本次作业完整实现了"数据加载 → 网络设计 → 前馈 → 反向传播 → 损失/正则 → 多种优化器 → 训练/验证/测试评估"的图像分类全流程，所有核心计算均由 NumPy 手写。 实验复现了深度学习中的经典

结论：交叉熵是分类任务的首选损失；正则化需按 模型规模/训练程度适度选择；mini-batch + 动量/自适应优化（Momentum、Adam） 是兼顾精度与效率的标准训练方案。

9. 代码与复现

```
mlp_numpy/
├── mlp.py                # 核心库：数据加载、网络前向/反向、损失、优化器、训练循环
├── run_experiments.py    # 实验入口：python run_experiments.py all
├── make_report.py        # 从 results/results.jsonl 生成全部图表
├── results/results.jsonl# 全部实验的原始记录（损失/精度曲线、耗时）
└── results/figs/         # 本文全部插图
```

复现步骤：`python run_experiments.py all && python make_report.py`（MNIST 会自动下载到 `./data`）。